

PRÁCTICA 1 — FUERZA BRUTA Y BACKTRACKING

Análisis y Diseño de Algoritmos

Integrantes

Viviana Arango Tabares
Manuela Buriticá Guzmán
Juan José Roldán

Curso

Análisis y Diseño de Algoritmos

Profesor

Carlos Alberto Álvarez Henao

Fecha

30 de Agosto del 2026
Universidad EAFIT
Medellín
2026

1. Introducción

En esta práctica se estudian e implementan los algoritmos de **Fuerza Bruta** y **Backtracking** para resolver problemas de búsqueda sobre diferentes espacios de soluciones. El propósito es comprender su funcionamiento y analizar cómo cambia su costo computacional al aumentar el tamaño del problema. La práctica incluye el modelamiento y diseño de los algoritmos, su implementación, pruebas y experimentación, para finalmente comparar su comportamiento teórico y empírico.

El documento se organiza de la siguiente manera: primero se presenta el contexto y la fundamentación del problema, seguido del modelamiento y diseño de los algoritmos. Luego se presentan el pseudocódigo, la implementación, el análisis de complejidad, los casos de prueba y la experimentación. Finalmente, se analizan los resultados, se comparan las estrategias y se presentan las conclusiones.

2. Contexto del problema

El problema consiste en explorar diferentes espacios de soluciones mediante estrategias de búsqueda. En Fuerza Bruta, se busca una contraseña a partir de un hash SHA-256 generando sistemáticamente cadenas de una longitud determinada sobre los alfabetos A1 y A2. En Backtracking, se construyen contraseñas de forma progresiva buscando cumplir restricciones de composición, como cantidades mínimas de letras, mayúsculas, dígitos y símbolos, además de evitar caracteres consecutivos iguales.

Ambas estrategias permiten abordar el problema de manera sistemática, pero Backtracking puede descartar durante la construcción aquellas ramas que ya no pueden cumplir las restricciones. Esto permite estudiar posteriormente la diferencia entre una exploración sin poda y otra que utiliza información parcial para reducir la búsqueda.

3. Fundamentación teórica

Para resolver estos problemas se utilizan estrategias de búsqueda exhaustiva y recursiva. La Fuerza Bruta consiste en probar sistemáticamente las posibles alternativas hasta encontrar una solución o agotar el espacio de búsqueda.

Backtracking parte de una solución parcial y explora sus posibles continuaciones de forma recursiva; cuando una elección ya no resulta adecuada, se abandona esa rama y se continúa con otra posibilidad. De esta forma, puede reducir la búsqueda cuando existen condiciones que permiten identificar tempranamente soluciones parciales que no son válidas.

En esta práctica, estas ideas se aplican directamente a las instancias definidas para cada algoritmo: Fuerza Bruta recorre los candidatos de los alfabetos establecidos y compara sus hashes, mientras que Backtracking construye contraseñas verificando progresivamente las restricciones de la política.

4. Modelamiento

4.1. Modelamiento de Fuerza Bruta

Alfabetos:

- A1 = {a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,z} (Tamaño = 26) con longitud de n, donde esta pertenece a {3,4,5,6}.
- A2 = {a,b,c,d,e,f,g,h,i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,z,0,1,2,3,4,5,6,7,8,9} (Tamaño = 36) como longitud de n, donde esta pertenece a {3,4,5}

Espacio de búsqueda:

- **A1: Tiene un espacio de búsqueda** $|\Sigma|^n = 26^n$, donde n es la longitud de entrada.
- **A2: Tiene un espacio de búsqueda** $|\Sigma|^n = 36^n$, donde n es la longitud de entrada.

Para ambos espacios de búsqueda cada uno de los candidatos se genera exactamente una vez.

Instancias creadas:

- **Alfabeto: A1 - Longitud(n): 4 - Clave: ijax**
Hash: c56d1270821c92b3205d36e89d1fb0fd0bad3b9f2862242d6b2a5a3392a6b499
- **Alfabeto: A2 - Longitud(n): 4 - Clave: ans9**
Hash: b55df863e210fa4784207934f751b2342a6f1f59f63c5f0fabbca7a9f9fba7fa

- **Alfabeto:** A1 - **Longitud(n):** 5 - **Clave:** uxyby
Hash: 3d703d21cf2e0792a465caf7e59b6e80b65fe53d90cea94fab66e40ebf663242
- **Alfabeto:** A2 - **Longitud (n):** 5 - **Clave:** r0la7
Hash: 199d6d898dfc43886d05ba5540f3da1e42d56c66327de37f67fc981889962c1d
- **Alfabeto:** A1 - **Longitud(n):** 6 - **Clave:** wpctwv
Hash: 1bc54ae0767a4dba59cb26bb16c4148d058d9a0d7b628340576f4c054432b96d

Dado un alfabeto Σ (A1 o A2), una longitud n y un hash SHA-256 en representación hexadecimal con objetivo H , el algoritmo genera sistemáticamente todas las cadenas de longitud n pertenecientes a Σ^n . Para cada candidato calcula su hash y lo compara con H . Si coinciden, se devuelve el candidato; si se agota el espacio, se informa que no fue encontrado.

4.2. Modelamiento de Backtracking

- **Alfabeto:** el mismo alfabeto de la Sección 9.2, compuesto por 26 minúsculas, 26 mayúsculas, 10 dígitos y 5 símbolos, para un total de 67 caracteres.
- **Política de contraseñas:** cada instancia se define con una longitud fija n y cuatro mínimos por categoría (minLower, minUpper, minDigit, minSymbol), además de la prohibición de dos caracteres idénticos consecutivos, aplicada a todas las instancias.
- **Estado parcial:** un prefijo de longitud k ($0 \leq k \leq n$) junto con los contadores de cuántos caracteres de cada categoría (minúsculas, mayúsculas, dígitos, símbolos) contiene ese prefijo, y el último carácter agregado (necesario para verificar la restricción de no consecutivos).
- **Estado inicial:** el prefijo vacío ($k = 0$), con todos los contadores en cero.
- **Estados terminales:** los prefijos de longitud n . Un estado terminal es solución si sus contadores satisfacen los cuatro mínimos de la política.
- **Espacio de búsqueda:** el árbol de decisión tiene, en el peor caso, tamaño $\sum_{k=0}^n |\Sigma|^k$, donde $|\Sigma| = 67$. Ese es el tamaño que recorre la versión sin poda la versión con poda solo visita el subconjunto de ese árbol que aún puede llegar a una solución válida.
- **Instancias creadas:** se definieron dos grupos. El primero, usado para verificar la correctitud del algoritmo, usa el alfabeto real con valores de n pequeños para que ambas versiones con y sin poda puedan terminar de recorrerse por completo. El segundo grupo corresponde a las instancias oficiales derivadas de la semilla del equipo; la instancia de referencia y las cinco variantes completa_n8, completa_n6, completa_n10,relajada_n8 y sinrestr_n6,

5. Diseño algorítmico

5.1. Diseño de Fuerza Bruta

5.1.1 Definición de entradas y salida

Entrada:

- **hash_objetivo:** Cadena hexadecimal de 64 caracteres.
- **alfabeto:** Conjunto de caracteres válidos (A1 o A2).
- **longitud:** Valor entero n que define la longitud de la contraseña a probar.

Salida:

- **Mensaje:**
Mensaje "Encontrada" si se encuentra una coincidencia dentro del espacio de búsqueda.
Mensaje "No encontrada" si se explora todo el espacio Σ^n y no se encuentra coincidencia alguna.
- Número total de evaluaciones hecho en el espacio de búsqueda
- Tiempo de ejecución

5.1.2 Definición de las restricciones

- Alfabetos soportados: A1 y A2.
- Longitudes soportadas: A1 donde n pertenece a $\{3, 4, 5, 6\}$, A2 donde n pertenece a $\{3, 4, 5\}$.
- Los candidatos son cadenas de longitud n .
- La comparación se realiza mediante SHA-256.
- El experimento se ejecuta sobre contraseñas sintéticas.
- No se utilizan credenciales o sistemas reales.

5.1.3 Diseño del algoritmo

El algoritmo construye el candidato posición por posición. Para cada posición se prueban todos los caracteres disponibles en el alfabeto. Cuando se alcanza la longitud n , se calcula el SHA-256 del candidato y se compara con el hash objetivo. Si existe coincidencia, se detiene la búsqueda; de lo contrario, continúa hasta recorrer todo el espacio.

5.2. Diseño de Backtracking

Entrada y Salida

Entrada:

- alfabeto: los 67 símbolos definidos en la Sección 9.2.
- n : longitud fija de la contraseña.
- minLower, minUpper, minDigit, minSymbol: mínimos exigidos de cada categoría.

Salida:

- número total de soluciones encontradas.
- hasta 20 contraseñas de ejemplo, guardadas para inspección y verificación cruzada contra la política.
- número de nodos visitados durante la búsqueda.
- tiempo de ejecución.

Restricciones

- Longitud fija n para toda la instancia.
- Mínimos por categoría: minLower, minUpper, minDigit, minSymbol.
- Prohibición de dos caracteres idénticos consecutivos.
- El alfabeto es fijo: 67 símbolos (26 minúsculas, 26 mayúsculas, 10 dígitos, 5 símbolos especiales).
- El experimento se ejecuta sobre políticas sintéticas; no se generan ni almacenan contraseñas reales de ningún sistema.

Diseño de Algoritmo

El algoritmo construye la contraseña de forma incremental, carácter por carácter, igual que la enumeración de Fuerza Bruta, pero antes de descender a cada rama evalúa dos condiciones de poda:

Poda por caracteres consecutivos: si el carácter candidato es igual al último carácter del prefijo actual, la rama se descarta de inmediato, sin generar el hijo.

Poda por factibilidad de mínimos: con los contadores actualizados (cuántas minúsculas, mayúsculas, dígitos y símbolos tendría el prefijo si se agrega el carácter candidato), se calcula cuánto falta de cada categoría para cumplir

la política. Si la suma de lo que falta supera el número de posiciones que quedan por llenar, ninguna extensión de esa rama puede ser válida, y se descarta sin explorarla.

Cuando el prefijo alcanza la longitud n sin haber sido descartado, se verifica una última vez contra la política completa (como control de correctitud) y, si cumple, se cuenta como solución. Esta misma lógica, sin ninguna de las dos podas, es la que implementa la versión sin poda usada como comparación en la Sección 8.2: recorre el árbol completo y filtra únicamente al llegar a las hojas.

6. Pseudocódigo

6.1. Pseudocódigo de Fuerza Bruta

```
Escribir "Ingrese el Hash Objetivo (SHA-256):"
Leer hash_objetivo
Escribir "Seleccione Alfabeto (A1 para minúsculas, A2 para alfanumérico):"
Leer opcion_alfabeto
Si opcion_alfabeto == "a1" O opcion_alfabeto == "A1" Entonces:
    alfabeto = "abcdefghijklmnopqrstuvwxyz"
Sino Si opcion_alfabeto == "a2" O opcion_alfabeto == "A2" Entonces:
    alfabeto = "abcdefghijklmnopqrstuvwxyz0123456789"
Sino:
    Escribir "Opción no válida."
    Retornar
FinSi
Escribir "Ingrese la longitud (n):"
Leer longitud
Si longitud no pertenece a las longitudes permitidas para el alfabeto Entonces:
    Escribir "Longitud no válida para el alfabeto seleccionado."
    Retornar
FinSi
tiempo_inicio = IniciarTemporizador()
(candidato, evaluaciones, estado) =
    FuerzaBruta(hash_objetivo, alfabeto, longitud)
tiempo_fin = FinalizarTemporizador()
tiempo_ejecucion = tiempo_fin - tiempo_inicio
Escribir " RESULTADO"
Escribir "Estado: ", estado
Si estado == "ENCONTRADA" Entonces:
    Escribir "Candidato: ", candidato
FinSi
Escribir "Evaluaciones: ", evaluaciones
Escribir "Tiempo de ejecución: ", tiempo_ejecucion
Algoritmo FuerzaBruta(hash_objetivo, alfabeto, longitud):
```

candidato = cadena vacía de tamaño longitud

evaluaciones = 0

Procedimiento Resolver(posicion):

Si posicion == longitud Entonces:

 evaluaciones = evaluaciones + 1

 hash_candidato = SHA256(candidato)

 Si hash_candidato == hash_objetivo Entonces:

 Retornar VERDADERO

 FinSi

 Retornar FALSO

FinSi

Para cada caracter ch en alfabeto Hacer:

 candidato[posicion] = ch

 Si Resolver(posicion + 1) Entonces:

 Retornar VERDADERO

 FinSi

FinPara

Retornar FALSO

FinProcedimiento

encontrado = Resolver(0)

Si encontrado Entonces:

 Retornar (candidato, evaluaciones, "ENCONTRADA")

Sino:

 Retornar (NULO, evaluaciones, "NO ENCONTRADA")

FinSi

FinAlgoritmo

6.2. Pseudocódigo de Backtracking

Leer n, minLower, minUpper, minDigit, minSymbol

alfabeto = "a..z A..Z 0..9 ! @ # \$ %"

prefijo = cadena vacia

soluciones = 0

ejemplos = lista vacia

Backtrack(prefijo, 0, 0, 0, 0)

Escribir "Soluciones encontradas: ", soluciones

Procedimiento Backtrack(prefijo, contLower, contUpper, contDigit, contSymbol):

Si longitud(prefijo) == n Entonces:

 Si CumplePolitica(prefijo) Entonces:

 soluciones = soluciones + 1

 Si longitud(ejemplos) < 20 Entonces:

Agregar prefijo a ejemplos

FinSi

FinSi

Retornar

FinSi

restantes = n - (longitud(prefijo) + 1)

Para cada caracter c en alfabeto Hacer:

// Poda 1 caracteres consecutivos identicos

Si prefijo no es vacio Y ultimoCaracter(prefijo) == c Entonces:

Continuar

FinSi

(nl, nu, nd, ns) = ActualizarContadores(c, contLower, contUpper, contDigit, contSymbol)

faltanLower = maximo(0, minLower - nl)

faltanUpper = maximo(0, minUpper - nu)

faltanDigit = maximo(0, minDigit - nd)

faltanSymbol = maximo(0, minSymbol - ns)

// Poda 2 factibilidad de minimos

Si (faltanLower + faltanUpper + faltanDigit + faltanSymbol) > restantes Entonces:

Continuar

FinSi

Backtrack(prefijo + c, nl, nu, nd, ns)

FinPara

FinProcedimiento

Funcion CumplePolitica(cadena):

lower, upper, digit, symbol = contar por categoria en cadena

Para i desde 1 hasta longitud(cadena) - 1 Hacer:

Si cadena[i] == cadena[i-1] Entonces:

Retornar FALSO

FinSi

FinPara

Retornar (lower >= minLower) Y (upper >= minUpper) Y (digit >= minDigit) Y (symbol >= minSymbol)

FinFuncion

7. Implementación

7.1 Fuerza Bruta

El algoritmo de Fuerza Bruta fue implementado en C++ mediante una búsqueda recursiva basada en DFS. Para almacenar los resultados de la ejecución se definió la estructura **ResultadoFB**, que contiene el candidato encontrado, el número de evaluaciones realizadas y el estado de la búsqueda. La función **FuerzaBruta** construye las cadenas posición por posición mediante el procedimiento recursivo Resolver, recorriendo los caracteres disponibles en el alfabeto y calculando el hash SHA-256 de cada candidato completo mediante la biblioteca **picosha2**. El

contador evaluaciones permite registrar la cantidad de candidatos procesados y la búsqueda finaliza cuando se encuentra una coincidencia o cuando se recorre todo el espacio disponible. La función **ejecutarModulo()** se encarga de recibir el hash objetivo, seleccionar el alfabeto, solicitar la longitud, validar que los parámetros correspondan a las restricciones establecidas y ejecutar los módulos de Fuerza Bruta y Diccionario. Para la medición experimental, se utiliza la biblioteca **chrono** alrededor de la ejecución de Fuerza Bruta, permitiendo registrar su tiempo de ejecución. Finalmente, el programa incluye **AtaqueDiccionario** para realizar la búsqueda mediante un archivo de palabras.

7.2 Backtracking

El módulo se implementó en C++17, separando cada responsabilidad en su propio archivo: **bt_policy.hpp** define la política y el alfabeto, **bt_backtracking.cpp** trae el algoritmo con poda, **bt_sin_poda.cpp** la versión exhaustiva para comparar, y **bt_runner.cpp** corre las instancias y genera el CSV.

El prefijo se maneja como un **std::string** que crece y se deshace con **push_back/pop_back** en cada llamada recursiva (como la guía), y los contadores por categoría se pasan como parámetros en lugar de recalcularse en cada nodo, para no recorrer el prefijo completo cada vez que se evalúa la factibilidad.

Como el alfabeto real (67 símbolos) genera árboles de hasta 10^{18} nodos para algunas instancias de la , ambas versiones reciben un tope de nodos: al alcanzarlo, la ejecución se detiene y el resultado queda marcado como parcial en vez de reportar un número que podría confundirse con un recorrido completo. También se agregó una verificación cruzada: cada cadena que llega a longitud n se valida de nuevo contra la política completa antes de contarse como solución, como control de correctitud independiente de la poda. Para las instancias que no alcanzan a completarse, se usa además una fórmula cerrada ($\sum_{k=0}^n |\text{alfabeto}|^k$) que da el tamaño teórico del árbol, sin necesidad de recorrerlo.

8. Análisis de complejidad

8.1. Complejidad de Fuerza Bruta

Complejidad Temporal Teórica: El algoritmo explora un árbol de decisión de profundidad n con un factor de ramificación.

- **Mejor Caso $O(n)$:** Ocurre si el primer candidato coincide de inmediato con el hash objetivo.
- **Peor Caso $O(|\Sigma|^n \cdot n)$:** Ocurre cuando la clave no existe o se ubica en el último candidato lexicográfico, recorriendo las hojas del árbol y calculando el hash SHA-256 para cadenas de tamaño n .

Contraste con el Crecimiento Medido:

Los límites máximos del espacio de búsqueda son:

- **A1 ($n = 4$):** $26^4 = 456,976$
- **A2 ($n = 4$):** $36^4 = 1,679,616$
- **A1 ($n = 5$):** $26^5 = 11,881,376$
- **A2 ($n = 5$):** $36^5 = 60,466,176$
- **A1 ($n = 6$):** $26^6 = 308,915,776$

La tasa de procesamiento estuvo entre 120,000 y 197,000 hashes por segundo, confirmando un escalamiento de orden $O(|\Sigma|^n)$.

Complejidad Espacial: El algoritmo opera con una complejidad espacial de $O(n)$, correspondiente al nivel máximo de llamadas recursivas en la pila y al almacenamiento del arreglo de la cadena candidata actual de tamaño n .

8.2. Complejidad de Backtracking

Complejidad Temporal Teórica: el algoritmo con poda recorre un árbol de decisión de profundidad n , igual que Fuerza Bruta, pero descarta ramas antes de explorarlas cuando no van por buen camino

Mejor Caso: ocurre cuando la suma de mínimos es igual o cercana a n , de modo que la poda descarta la mayoría de las ramas desde niveles muy tempranos. Se observó en `completa_n6`, donde la suma de mínimos (8) supera la longitud disponible (6): el algoritmo visita un único nodo y concluye de inmediato que no hay soluciones.

Peor Caso $O(|\Sigma|^n * n)$: ocurre cuando la política casi no impone restricciones (ej. `sinrestr_n6`), y el árbol recorrido se aproxima al tamaño del árbol completo $|\Sigma|^n$.

Caso Promedio: depende de qué tan ajustada esté la suma de mínimos respecto a n : mientras menos holgura haya, más temprano actúa la poda y menos nodos se visitan.

Contraste con el Crecimiento Medido: en las instancias de verificación que si terminaron. la poda redujo el espacio explorado en 85,91% y en 95,65%, coincidiendo exactamente el número de soluciones en ambas versiones (40.560 y 811.200), lo que confirma que la poda no descarta soluciones válidas. En las instancias oficiales, el árbol teórico crece entre $9,18 \times 10^{10}$ y $1,85 \times 10^{18}$ nodos demasiado grande para recorrerse por completo en un computador personal.

Complejidad Espacial: $O(n)$, por la profundidad de la pila de recursión y el almacenamiento del prefijo actual.

9. Casos de prueba

9.1. Semilla del equipo

Para generar las instancias propias del equipo se utilizaron los apellidos **Arango, Buritica y Roldan**, ordenados alfabéticamente. De acuerdo con el procedimiento establecido en la práctica, los apellidos se concatenaron en minúsculas y sin espacios ni tildes, obteniendo la cadena:

arangoburiticaroldan

Posteriormente, se sumaron los códigos ASCII de todos los caracteres de la cadena, obteniendo una suma de **2123**. Finalmente, se aplicó la operación módulo 100000:

Semilla = $2123 \bmod 100000 = 2123$

Por lo tanto, la semilla utilizada por el equipo fue **2123**, siendo determinante para reproducir las instancias utilizadas durante las pruebas.

9.2. Casos de prueba de Backtracking

9.2.1 Parámetros de Backtracking

A partir de la semilla **2123** se calcularon los parámetros de la política del equipo mediante las fórmulas establecidas en la Sección 8.2.

Parámetro	Valor
n	8
$minLower$	2
$minUpper$	2
$minDigit$	3
$minSymbol$	1
Caracteres consecutivos idénticos	No permitidos

Los valores iniciales fueron: $minLower = 4$, $minUpper = 2$, $minDigit = 3$ y $minSymbol = 1$. Como su suma era 10 y la longitud fija es $n = 8$, se redujo $minLower$ en 2 unidades, quedando los valores finales mostrados en la tabla.

¡El alfabeto utilizado está compuesto por 26 minúsculas, 26 mayúsculas, 10 dígitos y 5 símbolos (!, @, #, \$, %), para un total de 67 caracteres

9.2.2 Instancia de referencia

Antes de ejecutar las instancias propias del equipo se utilizó la instancia de referencia establecida en la práctica, con $n = 6$, $minLower = 2$, $minUpper = 1$, $minDigit = 1$, $minSymbol = 1$ y prohibición de caracteres idénticos consecutivos.

Durante la ejecución, la instancia de referencia alcanzó el límite de **50.000.000 de nodos** y fue marcada como incompleta. Por esta razón, las **46.087.993 soluciones encontradas no representan el total de soluciones de la instancia**, sino únicamente las encontradas antes de alcanzar dicho límite.

9.2.3 Instancias propias del equipo

Para evaluar diferentes niveles de dificultad se utilizaron las cinco variantes establecidas en el enunciado.

Instancia	Longitud	Parámetros
completa_n8	8	- <i>minLower</i>=2 - <i>minUpper</i>=2 - <i>minDigit</i>=3 - <i>minSymbol</i>=1
completa_n6	6	- <i>minLower</i>=2 - <i>minUpper</i>=2 - <i>minDigit</i>=3 - <i>minSymbol</i>=1
completa_n10	10	- <i>minLower</i>=2 - <i>minUpper</i>=2 - <i>minDigit</i>=3 - <i>minSymbol</i>=1
relajada_n8	8	Solo <i>minLower</i> =1
sinrestr_n6	6	Sin restricciones de composición

En todas las variantes se mantuvo la **prohibición de dos caracteres idénticos consecutivos**.

En particular, *completa_n6* es imposible porque los mínimos establecidos requieren $2 + 2 + 3 + 1 = 8$ caracteres, mientras que la longitud disponible es $n = 6$. Lo que hizo que el algoritmo con poda pudiera determinar que no existían soluciones.

9.2.4 Verificación de las pruebas

Los casos de prueba automatizados se utilizaron para comprobar diferentes propiedades del módulo de Backtracking. Se verificó que una política imposible produjera cero soluciones. Posteriormente, se compararon las versiones con poda y sin poda utilizando una instancia pequeña, comprobando que ambas encontraran la misma cantidad de soluciones y que la versión con poda visitara menos nodos.

Para el caso de prueba con el alfabeto reducido *abAB01!@*, $n = 5$ y un mínimo de un carácter para cada categoría, ambas versiones encontraron 6.144 soluciones. La versión con poda visitó 8.705 nodos, mientras que la versión sin poda visitó 37.449 nodos, obteniendo una reducción del 76,76 % en el espacio explorado.

También se verificó que los ejemplos generados por la versión con poda cumplieran las restricciones de la política y que no contuvieran caracteres idénticos consecutivos. Finalmente, las ejecuciones que alcanzaron los límites establecidos fueron identificadas explícitamente como parciales, evitando interpretar sus resultados como exploraciones completas.

9.3 Casos de prueba de Fuerza Bruta

9.3.1 Parámetros de Fuerza Bruta

Se utilizaron los dos alfabetos establecidos para la práctica. El alfabeto **A1** está compuesto por 26 letras minúsculas y admite longitudes $n \in \{3,4,5,6\}$. El alfabeto **A2** está compuesto por las 26 letras minúsculas y los 10 dígitos, para un total de 36 caracteres, y admite longitudes $n \in \{3,4,5\}$.

El espacio de búsqueda para **A1** corresponde a **26^n combinaciones** y para **A2** a **36^n** . Para cada ejecución, el algoritmo genera sistemáticamente las cadenas de la longitud indicada, calcula su hash SHA-256 y lo compara con el hash objetivo.

9.3.2 Instancia de Referencia

Para validación inicial, se utilizó la proporcionada en la práctica: la contraseña **abc12**, con el alfabeto **A2** y longitud **$n=5$** . El hash SHA-256 correspondiente es:
8d51feb34e3e69f6fa6dff577e2c60490cf9a7fcd835f9f6af1505b71d74773

La ejecución de Fuerza Bruta encontró correctamente la contraseña objetivo después de **50.249 evaluaciones**, con un tiempo de ejecución de **236,825 ms**. Esta prueba permitió verificar que la implementación funcionaba correctamente antes de ejecutar las instancias propias del equipo.

9.3.3 Instancias propias del equipo

Se generaron a partir de la semilla propia **2123**; se utilizaron 5 contraseñas objetivo con las longitudes 4, 4, 5, 5 y 6, alternando entre ambos alfabetos A1 y A2.

Instancia	Alfabeto	Longitud	Clave
objetivo_1	A1	4	ijax
objetivo_2	A2	4	ans9
objetivo_3	A1	5	uxyby
objetivo_4	A2	5	r0la7
objetivo_5	A1	6	wpctwv

Los hashes SHA-256 utilizados fueron los siguientes:

Instancia	SHA-256
objetivo_1	c56d1270821c92b3205d36e89d1fb0fd0bad3b9f2862242d6b2a5a3392a6b499
objetivo_2	b55df863e210fa4784207934f751b2342a6f1f59f63c5f0fabbca7a9f9fba7fa
objetivo_3	3d703d21cf2e0792a465caf7e59b6e80b65fe53d90cea94fab66e40ebf663242
objetivo_4	199d6d898dfc43886d05ba5540f3da1e42d56c66327de37f67fc981889962c1d
objetivo_5	1bc54ae0767a4dba59cb26bb16c4148d058d9a0d7b628340576f4c054432b96d

Estas 5 instancias corresponden a las definidas para el equipo y fueron utilizadas para la experimentación del módulo.

9.3.4 Verificación de las pruebas

Para comprobar el funcionamiento del módulo se realizaron pruebas unitarias sobre distintos escenarios. Primero se verificó un caso exitoso y uno fallido de Fuerza Bruta utilizando el alfabeto A1. Después se añadió una prueba con el alfabeto A2 para comprobar el manejo de letras y dígitos.

También se verificó el funcionamiento del ataque por diccionario mediante casos en los que la contraseña estaba presente y ausente del archivo, además de un caso con un archivo inexistente. Finalmente, se comprobó que el número de evaluaciones realizadas se registrara correctamente.

En total se ejecutaron **7 pruebas unitarias**, correspondientes a Fuerza Bruta exitosa en A1, Fuerza Bruta fallida, Fuerza Bruta con A2, ataque de diccionario exitoso, ataque de diccionario fallido, archivo de diccionario inexistente y conteo de evaluaciones. **Todas las pruebas pasaron correctamente.**

La instancia de referencia y las cinco instancias propias también fueron ejecutadas posteriormente para validar el comportamiento del algoritmo sobre casos reales de la práctica.

10. Experimentación

10.1. Experimentación de Fuerza Bruta

10.1.1 Tabla de resultados 1ra prueba

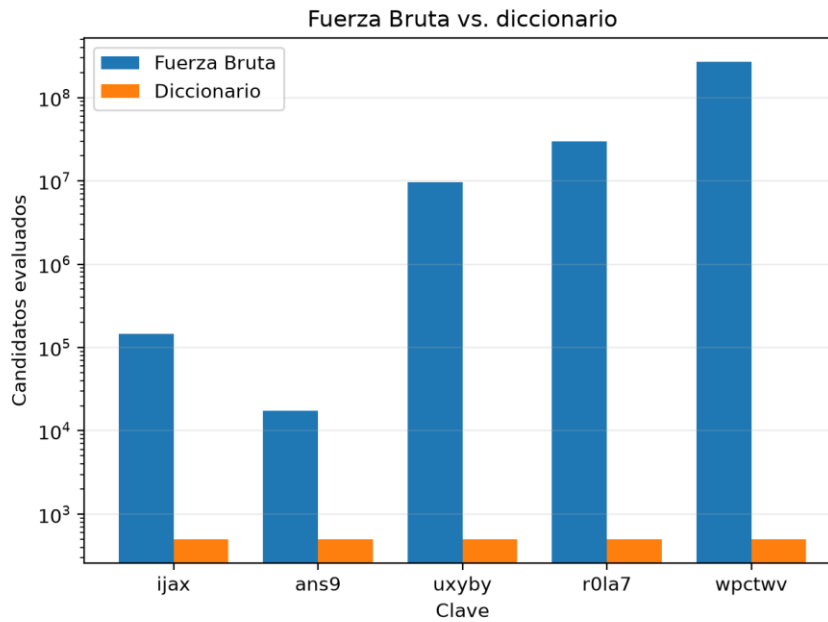
N	Contraseña	Alfabeto	Longitud(n)	Candidatos Evaluados	Tiempo (ms)
1	ijax	A1	4	146716	1196.51
2	ans9	A2	4	17532	131.66
3	uxyby	A1	5	9560043	72183.8
4	r0la7	A2	5	29780818	223951
5	wpctwv	A1	6	268293502	1.36E+06

10.1.2 Tabla de resultados 2da prueba (Diccionario oficial)

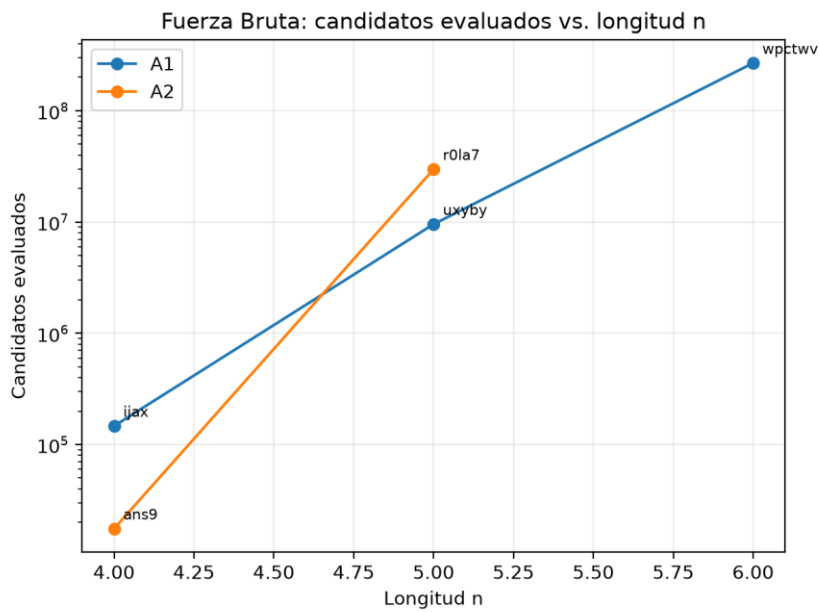
Instancia	Alfabeto	Longitud (n)	Clave	Evaluaciones diccionario	Estado
Referencia	A2	5	abc12	500	NO ENCONTRADA
objetivo_1	A1	4	ijax	500	NO ENCONTRADA
objetivo_2	A2	4	ans9	500	NO ENCONTRADA
objetivo_3	A1	5	uxyby	500	NO ENCONTRADA
objetivo_4	A2	5	r0la7	500	NO ENCONTRADA
objetivo_5	A1	6	wpctwv	500	NO ENCONTRADA

10.1.3 Gráficas de resultados

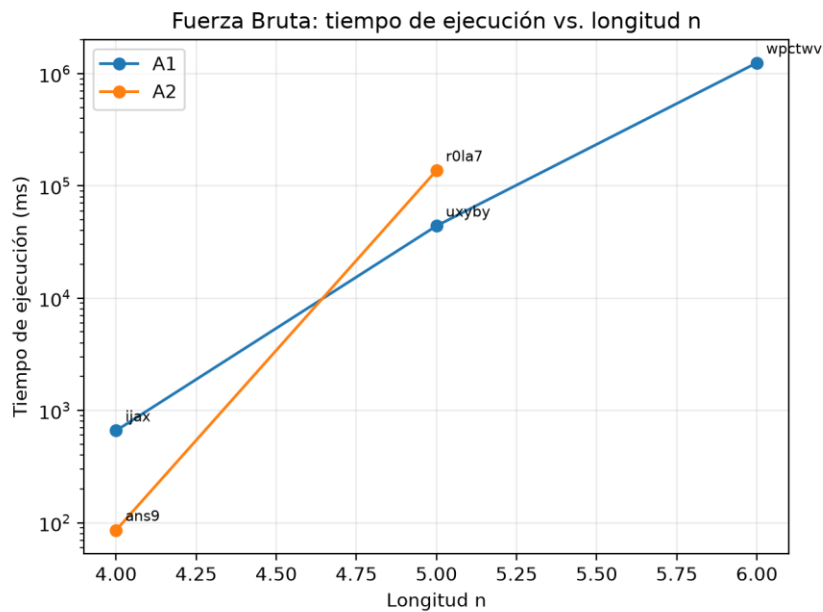
- **Figura 1:** Fuerza Bruta vs. diccionario



- **Figura 2:** Candidatos evaluados vs. Longitud n



- **Figura 3:** Tiempo de ejecución vs. Longitud n



10.2. Experimentación de Backtracking

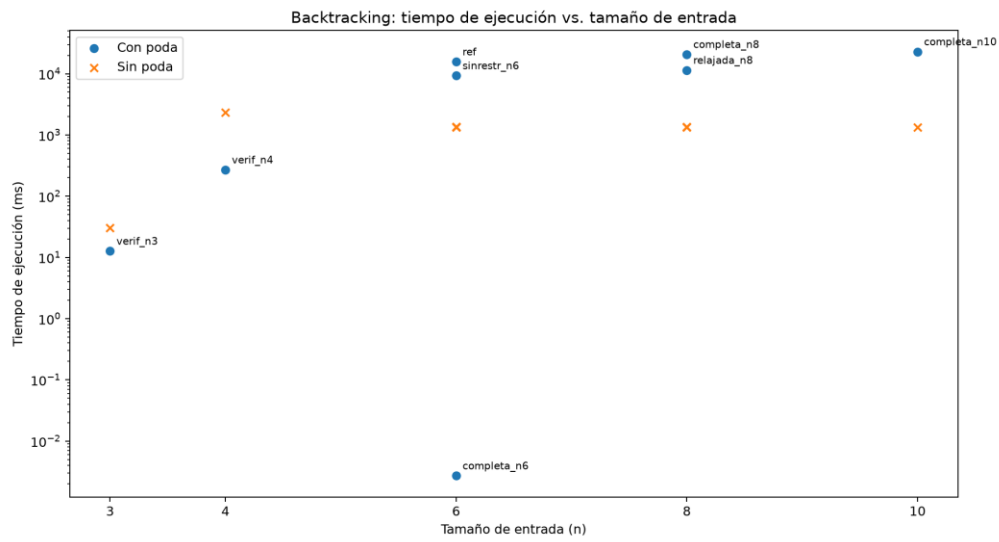
10.2.1 Tabla de resultados

Instancia	Tipo	n	minLower	minUpper	minDigit	minSymbol	Nodos con poda	Poda completa
verif_n3	verificación	3	1	1	1	0	43015	Si
verif_n4	verificación	4	1	1	1	1	890720	Si
ref	oficial	6	2	1	1	1	50000000	No
completa_n8	oficial	8	2	2	3	1	50000000	No
completa_n6	oficial	6	2	2	3	1	1	Si
completa_n10	oficial	10	2	2	3	1	50000000	No
relajada_n8	oficial	8	1	0	0	0	50000000	No
sinrestr_n6	oficial	6	0	0	0	0	50000000	No

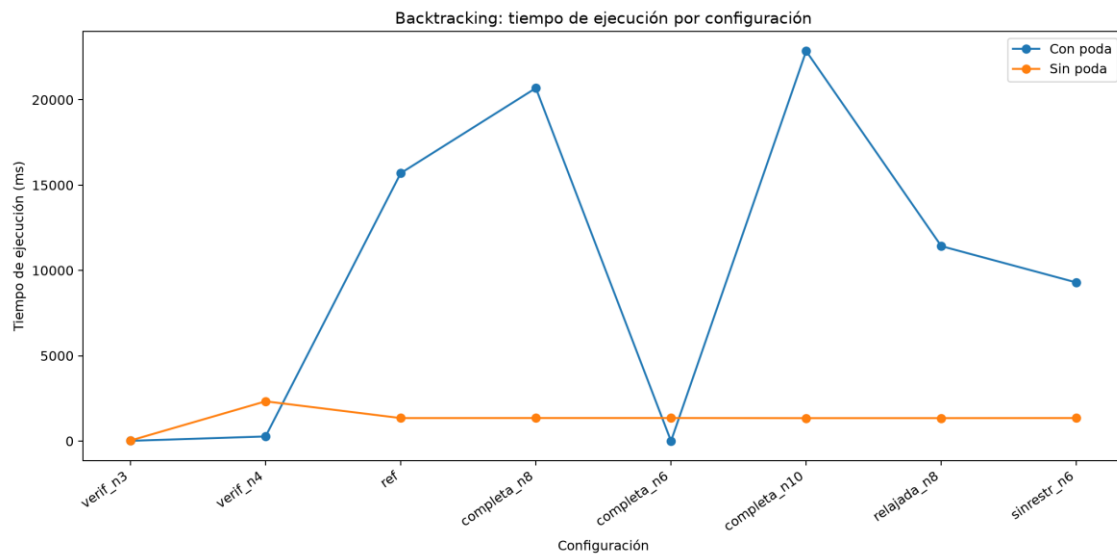
Tiempo con poda (ms)	Nodos sin poda	Sin poda completa	Tiempo sin poda (ms)	Nodos teóricos sin poda	Soluciones con poda	Soluciones sin poda	Soluciones coinciden	Reducción (%)
12,686	305320	Si	30,5804	305320	40560	40560	Si	85,911503
269,481	20456441	Si	2327,36	20456400	811200	811200	Si	95,645772
15698,7	20000000	No	1345,94	91829000000	46087993	0	N/A	N/A
20684,5	20000000	No	1349,27	4,1222E+14	44692669	0	N/A	N/A
0,002708	20000000	No	1349,65	91829000000	0	0	N/A	N/A
22853,4	20000000	No	1342,84	1,85046E+18	44692667	0	N/A	N/A
11423,3	20000000	No	1343,75	4,1222E+14	49242418	0	N/A	N/A
9292	20000000	No	1348	91829000000	49242420	0	N/A	N/A

10.2.2 Gráficas de resultados

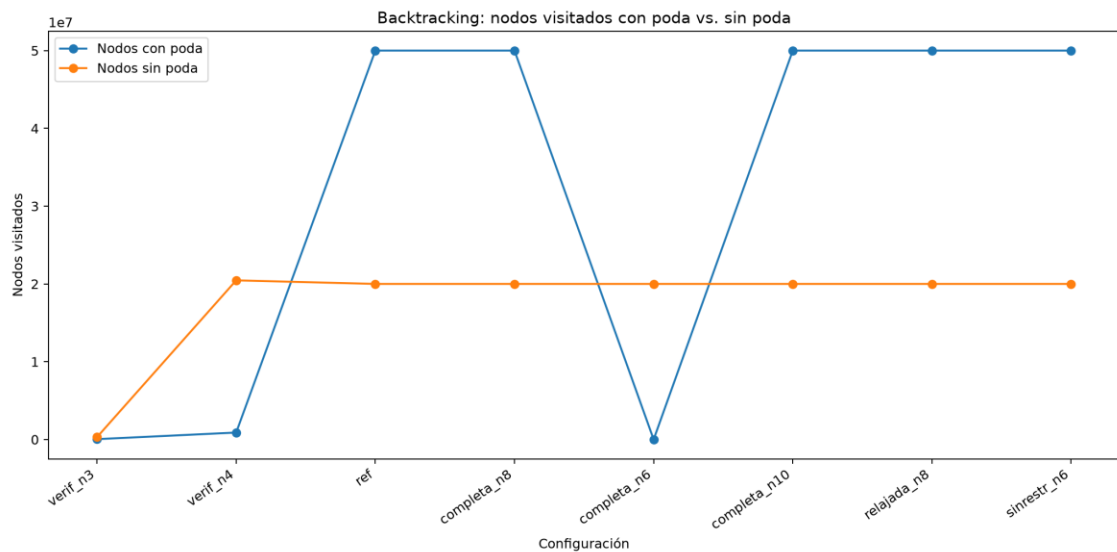
- **Figura 1:** Tiempo de ejecución vs. tamaño de entrada (n)



- **Figura 2:** Tiempo de ejecución por configuración



- **Figura 3:** Nodos visitados con poda vs. Sin poda



11. Resultados

11.1 Módulo de Backtracking

Las pruebas realizadas sobre el módulo de Backtracking permitieron obtener resultados para las diferentes configuraciones evaluadas. En las instancias de verificación que finalizaron completamente, para $n=3$ se obtuvieron **40.560 soluciones**, con 43.015 nodos visitados utilizando poda y 305.320 sin poda. Para $n=4$, ambas versiones encontraron **811.200 soluciones**, mientras que la versión con poda visitó 890.720 nodos frente a 20.456.441 de la versión sin poda.

En las configuraciones de mayor tamaño, **varias ejecuciones alcanzaron los límites de nodos establecidos y fueron marcadas como parciales**. En particular, la instancia de referencia alcanzó 50.000.000 de nodos con poda y 20.000.000 sin poda. De igual manera, las configuraciones con $n=8$ y $n=10$ no pudieron completarse dentro de los límites definidos.

Un caso particular fue completa_n6, cuya política requiere como mínimo 8 caracteres para una contraseña de longitud 6. En consecuencia, la versión con poda encontró 0 soluciones después de visitar un único nodo, mientras que la versión sin poda alcanzó su límite de 20 millones de nodos.

11.2 Módulo de Fuerza Bruta

Las pruebas realizadas sobre el módulo de Fuerza Bruta permitieron obtener resultados para las diferentes configuraciones evaluadas. En las instancias de menor tamaño, para $n = 4$ con el **alfabeto A1** se evaluaron **146.716 candidatos**, mientras que con el **alfabeto A2** se evaluaron **17.532 candidatos**. Los tiempos de ejecución registrados fueron de 1.196,51 ms para A1 y 131,66 ms para A2, respectivamente.

Al aumentar la longitud a $n = 5$, se observó un incremento considerable en el número de candidatos evaluados. Para **A1** se evaluaron **9.560.043 candidatos**, con un tiempo de ejecución de 72.183,8 ms, mientras que para **A2** se evaluaron **29.780.818 candidatos**, alcanzando un tiempo de 223.951 ms. Esto evidencia el aumento del costo computacional al utilizar alfabetos de mayor tamaño y longitudes superiores.

En la instancia de mayor tamaño evaluada, correspondiente a A1 con $n = 6$, el algoritmo evaluó **268.293.502 candidatos** antes de encontrar la contraseña objetivo, con un tiempo de ejecución aproximado de $1,36 \times 10^6$ ms. Esta instancia presentó el mayor costo computacional de las pruebas realizadas.

Los resultados también muestran que el número de evaluaciones no depende únicamente del tamaño total del espacio de búsqueda, sino de la posición de la contraseña objetivo dentro del orden en que el algoritmo genera los candidatos. Por esta razón, dos instancias con el mismo alfabeto y longitud pueden presentar cantidades diferentes de evaluaciones y tiempos de ejecución.

12. Análisis de resultados

12.1 Módulo de Backtracking

Los resultados muestran un crecimiento rápido del costo al aumentar n . Entre $n=3$ y $n=4$, el tiempo con poda pasó de **12,686 ms a 269,481 ms**, mientras que sin poda pasó de **30,5804 ms a 2327,36 ms**. Esto es consistente con el crecimiento exponencial del espacio de búsqueda.

La poda tuvo un efecto significativo: para $n=3$ redujo los nodos explorados en **85,91 %** y para $n=4$ en **95,65 %**, manteniendo en ambos casos la misma cantidad de soluciones. Esto muestra que permite descartar ramas que no pueden producir soluciones válidas.

Para las configuraciones a partir de $n=6$, varias ejecuciones alcanzaron el límite de nodos y quedaron incompletas. Por ello, para este equipo y las condiciones utilizadas, **$n=6$ marca el punto a partir del cual el costo deja de ser manejable dentro de los límites establecidos**.

Finalmente, completa_n6 mostró claramente el beneficio de la poda: como los mínimos requerían 8 caracteres para una contraseña de longitud 6, la política era imposible y la versión con poda lo detectó visitando solo **1 nodo**, mientras que la versión sin poda llegó a su límite.

12.2 Módulo de Fuerza Bruta

Los resultados muestran un **crecimiento muy rápido del costo al aumentar n** , porque cada carácter nuevo multiplica el número de combinaciones posibles por el tamaño del alfabeto. Esto se ve, por ejemplo, en A1: al pasar de $n=4$ a $n=5$, las evaluaciones aumentaron de **146.716 a 9.560.043**. El comportamiento concuerda con el

crecimiento teórico de $O(2^n)$, pero el número exacto de evaluaciones depende de dónde se encuentre la clave dentro del orden de búsqueda.

También se observa que trabajar con un alfabeto más grande aumenta el costo. Para $n=5$, A2 requirió **29.780.818 evaluaciones**, frente a **9.560.043** en A1, ya que A2 tiene 36 símbolos mientras A1 tiene 26.

El caso de A1 con $n=6$ muestra el límite práctico del algoritmo: se realizaron **268.293.502 evaluaciones** y el tiempo fue de aproximadamente **20,8 minutos**. A partir de este punto, el crecimiento deja de ser manejable para nuestro equipo. En conjunto, los resultados confirman experimentalmente que Fuerza Bruta tiene un crecimiento exponencial y que aumentar la longitud o el tamaño del alfabeto afecta bastante su costo.

13. Comparación algorítmica

13.1. Fuerza Bruta vs Diccionario

Ambos métodos buscan encontrar la contraseña a partir de su hash SHA-256, pero trabajan de manera diferente. Fuerza Bruta genera sistemáticamente los candidatos del espacio de búsqueda, mientras que el ataque por diccionario solo revisa las 500 palabras disponibles en *diccionario.txt*.

En las pruebas realizadas, Fuerza Bruta encontró las cinco claves propias, aunque necesitó desde **17.532 hasta 268.293.502 evaluaciones** dependiendo de la instancia. En cambio, el ataque por diccionario realizó únicamente **500 evaluaciones en cada caso**, pero no encontró ninguna de las claves. Esto se debe a que las contraseñas fueron generadas a partir de la semilla del equipo y no forman parte del diccionario proporcionado.

Por lo tanto, el diccionario resulta mucho más limitado en cantidad de trabajo, pero no garantiza encontrar la contraseña. Fuerza Bruta requiere un costo mucho mayor, pero garantiza encontrarla mientras pertenezca al espacio de búsqueda definido.

13.2. Backtracking con poda vs. sin poda

Ambas versiones recorren el mismo espacio de soluciones, pero la diferencia está en cómo lo hacen. La versión sin poda continúa explorando las combinaciones hasta completar cada cadena, mientras que la versión con poda puede detener una rama cuando ya se sabe que no podrá cumplir la política.

Esto se refleja en los resultados. Por ejemplo, para $n=4$, ambas versiones encontraron las mismas **811.200 soluciones**, pero con poda se visitaron **890.720 nodos**, mientras que sin poda se recorrieron **20.456.441**. Esto muestra que la poda permite llegar al mismo resultado haciendo mucho menos trabajo.

Por lo tanto, **la versión con poda es más conveniente cuando las restricciones permiten descartar ramas desde etapas tempranas**, mientras que la versión sin poda es más sencilla, pero termina explorando muchas combinaciones que no llevan a una solución válida.

14. Conclusiones

1. En las pruebas realizadas, se comprobó que la poda reduce considerablemente el espacio de búsqueda sin afectar las soluciones encontradas. También se observó que el costo aumenta rápidamente al incrementar la longitud de la entrada. Como tester, esto permitió verificar el correcto funcionamiento y la efectividad de la poda.
2. En las pruebas realizadas, se comprobó que el número de evaluaciones realizadas no necesariamente corresponde al tamaño completo del espacio de búsqueda, debido a que el algoritmo detiene la ejecución cuando encuentra una coincidencia. Por esta razón, la posición de la contraseña dentro del orden de generación influye directamente en el tiempo requerido.
3. En las pruebas realizadas, se comprobó que el costo de Fuerza Bruta aumenta al crecer el espacio de búsqueda. El algoritmo encontró correctamente todas las claves dentro del espacio definido, aunque el tiempo aumentó considerablemente en las entradas grandes. Por otro lado, el diccionario realizó como máximo 500 evaluaciones, pero no encontró ninguna de las claves objetivo, mostrando que es más limitado y depende de que la contraseña esté incluida en el archivo.
4. La poda por factibilidad resultó efectiva en políticas restrictivas con reducciones de 85-95% en las instancias de verificación, pero no cambia la cota exponencial en el peor caso: con políticas flexibles o alfabetos grandes, el backtracking sigue siendo, en la práctica, de similar costo a la fuerza bruta.

5. Con el alfabeto de 67 símbolos, $n=6$ ya marca el límite de lo que se puede recorrer completo en un computador personal este punto este punto es uno de los mas utiles de este trabajo.

15. Referencias

- cppreference.com. (2026). *assert – C++ reference* <https://en.cppreference.com/cpp/error/assert>
- GoogleTest. (2026). *GoogleTest Primer* <https://google.github.io/googletest/primer.html>
- GeeksforGeeks. (2025). *Brute Force Approach and its pros and cons.* <https://www.geeksforgeeks.org/dsa/brute-force-approach-and-its-pros-and-cons/>
- Peleato, J. J. (s. f.). *Backtracking*. JJ Peleato Docs. <https://docs.jipeleato.com/algoritmia/backtracking>

16. Uso de herramientas de IA

Herramienta: ChatGPT (OpenAI)

Fecha: agosto de 2026

Uso: Se utilizó como apoyo para consultar conceptos de Fuerza Bruta y Backtracking, resolver dudas sobre el funcionamiento de las pruebas, interpretar resultados experimentales y recibir orientación para la organización de gráficas y redacción del informe.

16.1 Tester

Evidencia 1: Consulta realizada a ChatGPT sobre cómo realizar la suma ASCII



Evidencia 2: Consulta realizada a ChatGPT sobre el diseño de pruebas para Backtracking



Evidencia 3: Consulta realizada a ChatGPT sobre el diseño de pruebas para Fuerza Bruta



Evidencia 4: Consulta realizada a ChatGPT sobre el script para la realización de gráficas



Evidencia 5: Consulta realizada a ChatGPT sobre qué aspectos a tener en cuenta para el análisis de resultados

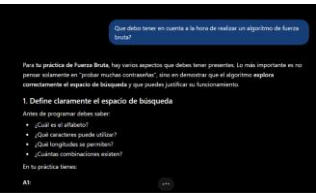


16.2 Fuerza Bruta

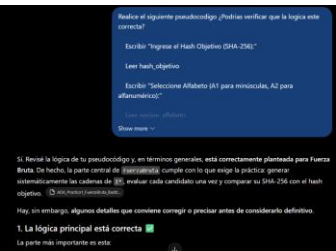
Evidencia 1: Consulta realizada a ChatGPT sobre detalles y entregas esperadas



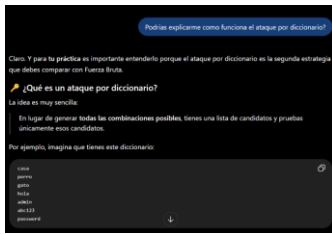
Evidencia 2: Consulta a ChatGPT sobre aspectos importantes de la fuerza bruta



Evidencia 3: Consulta realizada ChatGPT sobre la verificación de lógica del pseudocodigo

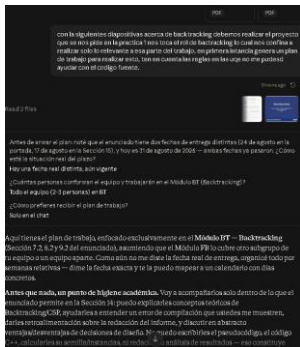


Evidencia 4: Consulta realizada a ChatGPT sobre dudas del concepto de ataque por diccionario



16.3 Backtracking

Consulta 1: Consulta sobre plan de trabajo



Evidencia 2: Consulta sobre existentes implementaciones ya sea pseudocódigo o código de backtracking en las diapositivas de clase



17. Contribución individual

- **Viviana Arango Tabares:** Responsable del testing de los módulos de Backtracking y Fuerza Bruta, incluyendo diseño y ejecución de pruebas, verificación de resultados y apoyo en la experimentación y análisis.
- **Manuela Buriticá Guzmán:** Persona responsable del modelado e implementación del algoritmo de la rama de Fuerza Bruta, realice y participe en su construcción, validación y evaluación experimental, buscando que la implementación y el desarrollo de la práctica fueran coherentes con los objetivos y requerimientos.
- **Juan José Roldán:** Responsable de la modelación e implementación del algoritmo de Backtracking.